```
            new MinimumAgeRequirement(18),
            new AllowedInLoungeRequirement()
        ));
});

services.AddSingleton<IAuthorizationHandler, MinimumAgeHandler>();
services.AddSingleton<IAuthorizationHandler, FrequentFlyerHandler>();
services.AddSingleton<IAuthorizationHandler, BannedFromLoungeHandler>();
services.AddSingleton<IAuthorizationHandler, IsAirlineEmployeeHandler>();
// Additional configuration
```

For this app, the handlers don't have any constructor dependencies, so I've registered them as singletons with the container. If your handlers have scoped or transient dependencies (the EF Core `DbContext`, for example), you might want to register them as scoped instead, as appropriate.

NOTE Services are registered with a lifetime of transient, scoped, or singleton, as discussed in chapter 9.

You can combine the concepts of policies, requirements, and handlers in many ways to achieve your goals for authorization in your application. The example in this section, although contrived, demonstrates the components you need to apply authorization declaratively at the action method or Razor Page level by creating policies and applying the `[Authorize]` attribute as appropriate.

As well as applying the `[Authorize]` attribute explicitly to actions and Razor Pages, you can configure it globally, so that a policy is applied to every endpoint in your application. Additionally, for Razor Pages you can apply different authorization policies to different folders. You can read more about applying authorization policies using conventions in Microsoft's "Razor Pages authorization conventions in ASP.NET Core" documentation: http://mng.bz/nMm2.